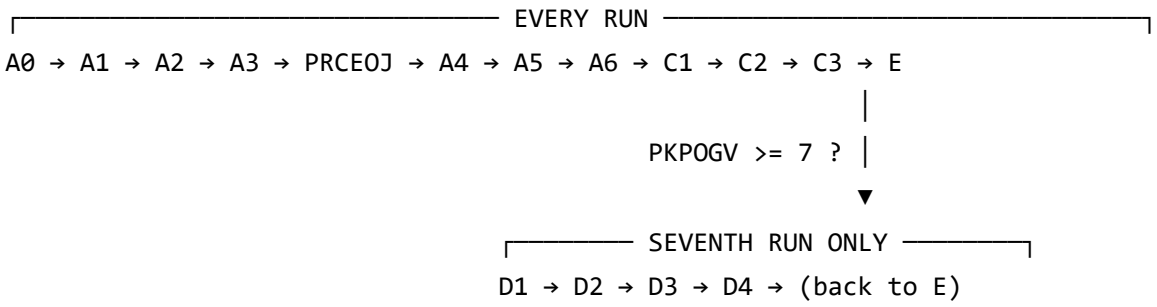
# Meijer POS Pipeline — Process Flow & Table Map

What happens at each stage of a run, and which table each stage touches. Phase names and ordering are taken from `PipelinePhase` and the call order in `PosPipelineOrchestrator`, not from the design docs — this is what the code does.

| | |
|---|---|
| Orchestrator | `ReaderLink.Application/Pipeline/PosPipelineOrchestrator.cs` |
| Phases | 15 ( `A0` – `A6` , `C1` – `C3` , `D1` – `D4` , `E` ) |
| Schemas | `pos` (15 tables + 1 staging) · `ctl` (2) · `ref` (16) |

## The shape of a run

Every transmission runs the **A** and **C** phases. Only the **seventh** transmission of a POS week continues into **D** — in the same process, with no separate trigger.




The whole A→E span runs inside **one transaction** on one shared connection. Failure anywhere rolls back every business write and marks the ledger `'F'` .

# Stage by stage

## A0 — Guard · `pos.RUN_LEDGER`

Allocates the RunId and decides whether this transmission may proceed at all.

`INSERT Status='A'` **outside** the transaction. The filtered unique index `UQ_RUN_LEDGER_DupKey` on `(AgencyCode, ChainCode, DupCheckKey) WHERE Status IN ('A','C')` **is** the guard — a repeat presentation violates the index rather than being caught by a race-prone `SELECT` -then- `INSERT`. On violation the caller writes a `Status='R'` refusal row pointing at the blocker and the run exits.

`DupCheckKey` = SHA-256 over `"{AgencyChain}|{Container}/{BlobName}"`. **The blob's container + name is the file's identity** — a renamed copy of identical bytes is a different key.

| Table | Access |
|---|---|
| `pos.RUN_LEDGER` | insert `'A'`, or insert `'R'` on a duplicate |

## A1 — Stage · `pos.POSDATAMJ` + `snapshots/`

Reads the inbound blob, bulk-copies it into the landing table, and writes an immutable snapshot.

Records must already be **exactly 512 bytes** — A1 refuses a badly framed file rather than tolerating it, because field-level tolerance belongs at A3, inside an already-framed record. Legacy's equivalent is POSADDTLR's `FPOSDATAT IP F 512 DISK`: a DB2 physical file GENTRAN wrote and the RPG cycle read. The staging table reproduces that contract; the blob is the modern substitute for GENTRAN's write.

| Table / store | Access |
|---|---|
| `pos.POSDATAMJ` | bulk insert, one row per 512-byte record, keyed `(FileId, Seq)` + `RunId` |
| `snapshots/` (blob) | write-once archive, independent of the working copy |

## A2 — Clear captures · `pos.POS850M`

Runs, and deletes nothing. Legacy cleared the chain's prior unmatched-SKU capture; under the no-delete rule `RunId` leads the PK, so a fresh RunId is already a clean capture scope. The phase executes so an auditor sees it happen rather than inferring a gap from an absence.

| Table | Access |
|---|---|
| `pos.POS850M` | none — contractually zero-delete |

## A3 — Parse & classify · reads `pos.POSDATAMJ`

Reads the staged rows **back out of the table** rather than reusing an in-memory copy, so parsing operates on what A1 actually wrote. Turns 512-byte positional records into a typed segment stream, then flattens every non-blank `SDQ` store/quantity pair into a **fact**.

Facts are the real unit of work downstream: 89 for a 23-record fixture, **552,473** for a full transmission.

Also derives the run's two dates — week-ending Saturday and POS activity date (header date **minus one day** for Meijer).

| Table | Access |
|---|---|
| `pos.POSDATAMJ` | read, `RunId`-scoped |

## PRCEOJ — Receipt stamp · `ctl.POSCTLM`

Not a numbered phase; sits between A3 and A4. Stamps the transmission receipt onto the chain's control row — sequence number, timestamps, last-program.

| Table | Access |
|---|---|
| `ctl.POSCTLM` | update `PKLHMS, PKLNBR, PKLPRG, PKLTIM, PKLUSR` |

## A4 — Identity waterfall · `ref.* + pos.POSERRM / pos.POS850M`

Resolves each fact's barcode to a ReaderLink item by a **first-match-wins cascade over three horizons**, then checks postability.

```
UPCSAL18  (current cross-reference)   → hit? take SUITM# / SUCSKU / SUUPC
   ↓ miss
UPCSALH5  (prior-period history)      → hit? same three fields
   ↓ miss
WHSSCNM / SMPITSM  (barcode scan)     → hit, then reversed if never distributed
   ↓ miss
ERRCOD '01' UPC NOT FOUND → capture, continue
```

**The three identity columns that flow onward are reference output, not file content.** The inbound barcode is only the lookup key; what persists is `SUITM#` → item, `SUCSKU` → SKU, `SUUPC` → barcode. So unseeded reference data produces *no rows*, not blank columns.

An unresolved item is a **recorded business outcome, not an exception** — the transmission continues.

| Table | Access |
|---|---|
| `ref.UPCSALM` / `ref.UPCSALH` | read (warmed in-memory per run) |
| `ref.WHSSCNM` / `ref.SMPITSM` | read — scan fallback + never-distributed guard |
| `ref.WHSITMM` | read — item-master postability check |
| `pos.POSERRM` | insert — one reject row **per affected store slot**, not per item |
| `pos.POS850M` | insert — unmatched capture (zero rows on the Meijer path) |

## A5 — Post detail · `pos.POSDTL`

Posts resolved facts into the run's working detail: one row per **(agency, chain, store, item, activity-date)**, inserting on first occurrence and merging after.

On-hand, sold and on-order live in **disjoint columns** — one quantity type can never overwrite another's. Both accumulate on merge; replace-vs-accumulate is a *weekly* rule, not a daily one. A **zero on-hand is posted** (out-of-stock signal); a **zero sale is discarded** (non-event).

| Table | Access |
|---|---|
| `pos.POSDTL` | chunked `MERGE`, `PDSRC='D'` marking the daily slice |

## A6 — Store flip · `ref.POSRDSXSTR` → `pos.POSDTL`

Applies the store cross-reference, rewriting **only** agency, chain and store on a match and preserving every other value. One pass in arrival order, so flips don't cascade. For Meijer the target *asserts* no configuration exists rather than assuming it.

| Table | Access |
|---|---|
| `ref.POSRDSXSTR` / `ref.POSRDSXCHN` | read |
| `pos.POSDTL` | update on match |

## C1 — BookScan extract · `pos.POSNLSN` + `reports/`

Renders the day's sales into Nielsen's 20-byte positional file. Grouped by store: `92` header, `I3` details, `94` summary — per store group, with a final summary at EOF.

A detail line is emitted only for a **strictly positive** sold quantity, and excluded rows are excluded from the count and total. A store with no qualifying sale still gets a header and a **zero** summary.

The item→EAN conversion is evidence-derived: 10-digit items are `[flag][core9]` with flag `0` → `978` / `4` → `979` ; a 13-digit item with flag `2` carries a 12-digit UPC rendered with one trailing blank.

Then the send name is issued — `NLS` + a 5-digit control number from `pos.NLSNCTL` . **An empty extract consumes no number and produces no send.**

| Table / store | Access |
|---|---|
| `pos.POSDTL` | read, `PDSRC='D'` only |
| `pos.POSNLSN` | insert, `Seq` preserving byte order |
| `pos.NLSNCTL` (sequence) | `NEXT VALUE FOR` — one per non-empty run |
| `reports/` (blob) | write the rendered 20-byte file |

## C2 — Daily accumulate · `pos.POSDLYM` + `pos.POSHOLDM`

Adds the day to the chain's seven-day store, at **(agency, chain, store, week-ending, item, activity-code, activity-date)** grain — three inserts, one per activity bucket.

Guarded first by a **duplicate-activity-date probe**: if `pos.POSDLYM` already holds this activity date for the chain, the merge is skipped and the week counter must not advance (legacy's `Upd_Flag = NO` ).

A row whose seven-part key already exists is **parked, never merged and never discarded**. `P0NOID` is drawn from `pos.POSDLYSEQN` , once per inserted row per bucket.

| Table | Access |
|---|---|
| `pos.POSDLYM` | probe, then three `INSERT … WHERE NOT EXISTS` (QS/QA/QR) |
| `pos.POSHOLDM` | insert — parked duplicate keys |

## C3 — Week check · `ctl.POSCTLM`

Advances the arrival counter **once per distinct new activity date** — not once per file — and tests it against the rollover threshold. Reaching 7 is what triggers the D phases, in the same run.

| Table | Access |
|---|---|
| `ctl.POSCTLM` | update `PKPOGV` ; read `PKLSAT` for the weekly phases |

# D phases — seventh run only

## D1 — Week rebuild · `pos.POSDLYM` → `pos.POSDTL`

Clears the chain's working detail (scoped to the chain, other chains untouched) and rebuilds it from the seven-day store filtered to **exactly one week-ending date, taken from the control row**.

Here the merge semantics flip: **on-hand is replaced, sold is accumulated** — on-hand is a stock level so the latest wins; sold is a flow so it sums. Rebuilt rows are marked with a provenance discriminator so the day-grain and week-grain slices of `POSDTL` stay distinguishable.

| Table | Access |
|---|---|
| `pos.POSDLYM` | read, one week-ending date |
| `pos.POSDTL` | clear chain slice, then insert week-grain rows |

## D2 — Weekly edit & select · `pos.POSWRKW`

A fixed unconditional sequence: clear the work file, build it, report. Every store is validated against the store master — invalid-store sales are **tallied for the report and excluded**. Each sale is classified **planogram by default**, reclassified to **bestseller only on a positive card match**, with card selection using the week-ending date rather than the system date.

Exactly one work row per qualifying sale, carrying the full sold quantity and full on-hand.

| Table | Access |
|---|---|
| `ref.WHSWHSM` / `ref.POSBSHM` / `ref.POSBSPM` | read — store master, bestseller cards |

| Table | Access |
|---|---|
| `pos.STG_POSWRKW` | clear chain slice, insert one row per qualifying sale |

## D3 — History write · `pos.POSDTLM` / `POSSUMM` / `POSLTDM` / `POSVENM` / `POSCLLOG1`

Commits the week to permanent history, grouped by item — enriched once per item, then posted for every store row of that item.

**Detail history** replaces on-hand and accumulates sold. **Summary history** accumulates both, so on-hand sums across stores at chain/item/week grain. Life-to-date accumulates per row before aggregation, so a sale and an equal return still touch the record.

Nothing is ever deleted from these tables — they feed a change-data feed, which is why recovery is adjustment rather than delete-and-rerun. A repeated close is made idempotent by `pos.WEEKLY_MERGE_LEDGER`, and one close-audit row is written per chain (none for an empty week).

| Table | Access |
|---|---|
| `pos.POSDTLM` | merge — on-hand replaced, sold accumulated |
| `pos.POSSUMM` | merge — both accumulated |
| `pos.POSLTDM` / `pos.POSVENM` | accumulate life-to-date and vendor aggregates |
| `pos.POSCLLOG1` | insert — one close-audit row per chain |
| `pos.WEEKLY_MERGE_LEDGER` | idempotency guard for the whole close |
| `pos.POSCLERR1` | back-post exclusion — **zero rows** on the Meijer path |

## D4 — History companions · `ref.WHSITMM` + aggregates

Reconciles the on-sale date, and the summary-vs-detail on-hand drift.

On-sale reconciliation selects titles still awaiting confirmation that both sold and held stock this week, anchors on the **Tuesday strictly before** week-end, reconciles a publisher date within 28 days of that anchor, and stamps the chosen date and confirmed status onto the **item master, not history** — which removes the title from all future selections. This is the pipeline's only sanctioned write to `ref`.

The on-hand reconciliation writes only rows already in drift, and treats "nothing needed correction" as success rather than an error.

| Table | Access |
|---|---|
| `ref.WHSITMM` | **update** — the one audited reference write |
| `pos.POSSUMM` / `pos.POSDTLM` | read + corrective update on drift |

## E — Epilogue · `pos.RUN_LEDGER`

Closes the run. `UPDATE Status='C' + CompletedAtUtc` **inside** the transaction, so the close becomes visible only if the run commits. On failure the transaction rolls back, then `Status='F'` is written **outside** the transaction by the failure handler — and `'F'` does **not** block re-presentation.

| Table | Access |
|---|---|
| `pos.RUN_LEDGER` | update `'C'` (in-TX) or `'F'` (post-rollback) |

# Table map at a glance

| Schema | Table | Role | Written by |
|---|---|---|---|
| `pos` | `RUN_LEDGER` | RunId allocator + file-level idempotency guard | A0, E |
| `pos` | `POSDATAMJ` | 512-byte landing staging | A1 |
| `pos` | `POSDTL` | working detail — **two grains**, day and week | A5, A6, D1 |
| `pos` | `POSERRM` | identity / postability rejects | A4 |
| `pos` | `POS850M` | unmatched-SKU capture (zero rows for Meijer) | A2, A4 |
| `pos` | `POSNLSN` | rendered BookScan records | C1 |
| `pos` | `POSDLYM` | seven-day accumulation store | C2 |
| `pos` | `POSHOLDM` | parked duplicate keys | C2 |
| `pos` | `STG_POSWRKW` | weekly work file | D2 |

| Schema | Table | Role | Written by |
|---|---|---|---|
| pos | POSDTLM / POSSUMM | permanent history, detail + summary | D3 |
| pos | POSLTDM / POSVENM | life-to-date and vendor aggregates | D3 |
| pos | POSCLLOG1 / POSCLERR1 | weekly close audit / back-post exclusion | D3 |
| pos | WEEKLY_MERGE_LEDGER | weekly-close idempotency guard | D3 |
| ctl | POSCTLM | chain control row — cadence, counter, week key | PRCEOJ, C3 |
| ctl | EDICTLM | partner registry | read at dispatch |
| ref | 16 tables | master and cross-reference data | **read-only**, one exception (D4) |

# Three properties worth knowing

**Reference data is read-only.** All 16 `ref` tables are read-only to the pipeline, with the single audited exception of D4's on-sale-date stamp on the item master. They're warmed into memory per run, so a run sees a stable snapshot.

**`POSDTL` carries two grains.** A5 writes day-grain rows keyed on activity date; D1 writes week-grain rows keyed on week-ending Saturday. On the promoting Saturday those keys collide by construction, which is why a provenance discriminator is part of the primary key rather than a plain column.

**Nothing is deleted.** `POSDLYM`, `POSDTLM` and `POSSUMM` feed change-data replication. Every rerun protection is therefore a guard — the ledger, `NOT EXISTS`, the merge ledger — never a delete. Legacy achieved the same thing destructively, and every one of those mechanisms needed an explicit replacement.

*Derived from `PosPipelineOrchestrator`'s coded phase order and the DDL headers under `ReaderLink.Infrastructure/Database/Scripts/`. For per-column provenance see `Modernization/Parity/POSDLYM_COLUMN_JOURNEY.md` and `POSNLSN_EXTRACT_FILE_JOURNEY.md`.*